CEUB

EDUCAÇÃO SUPERIOR

Desenvolvimento de Sistemas

ceub.br

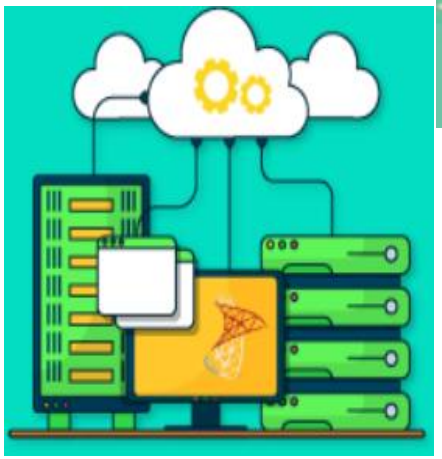
CEUB
EDUCAÇÃO SUPERIOR

Aula 1 - Fundamentos do Backend, POO e Ambiente

Agenda

- Desenvolvimento de aplicações
- Exemplos de aplicações
- Principais desafios do desenvolvimento
- Conceitos básicos de desenvolvimento web
- Linguagens de programação web
- Frameworks e bibliotecas para desenvolvimento web
- Conceitos básicos de desenvolvimento mobile
- Linguagens de programação mobile
- Frameworks e bibliotecas para desenvolvimento mobile
- Desenvolvimento de aplicações mobile nativas e híbridas
- Conceitos e fundamentos de segurança no desenvolvimento de software
- Controle de versão com Git: workflows, branching e boas práticas

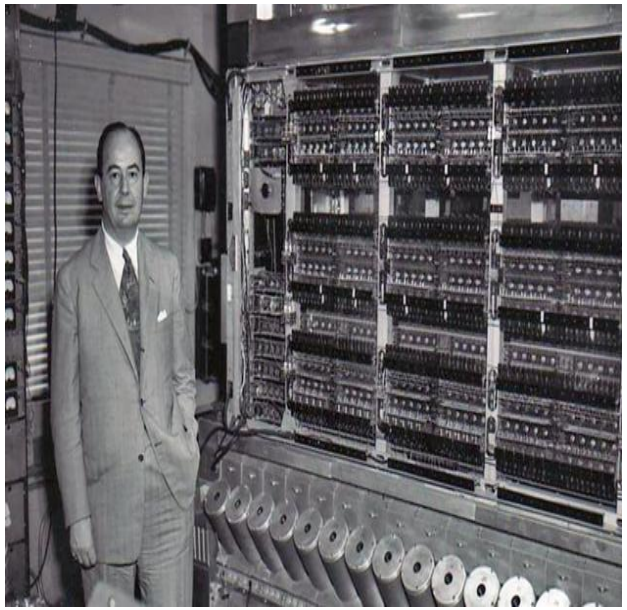
Introdução ao desenvolvimento Full Stack



Desenvolvimento de Sistemas

História

- Década de 40: Evolução dos Sistemas
- computacionais – Parte do investimento
- era designado principalmente ao Hardware;



História

Década de 60: Surgimento dos sistemas operacionais multiprogramação, possibilitou também o desenvolvimento de sistemas de grande porte (Crise do Software).

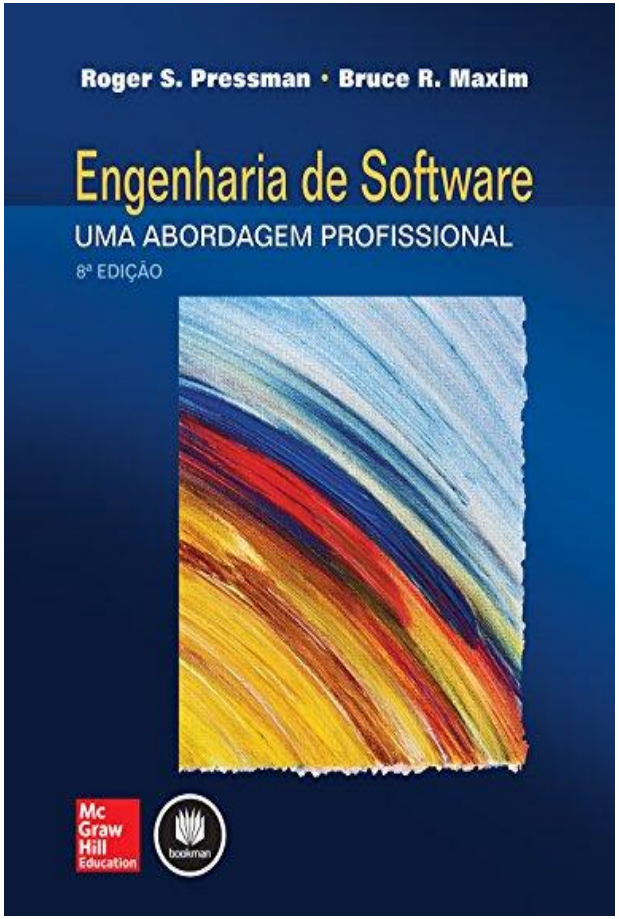
História

Início dos 70's: "Crise do Software"

- Engenharia de software praticamente inexistente
- Projetos estourando o orçamento
- Projetos estourando o prazo
- Software de baixa qualidade
- Software não satisfazendo os requisitos
- Projetos mal gerenciados e código difícil de manter

Processo de Construção de Software

O processo de software é representado por um conjunto sequencial de atividades, objetivos, transformações e eventos que integram estratégias para cumprimento da evolução de software (*Pressman*).



Engenharia de Software

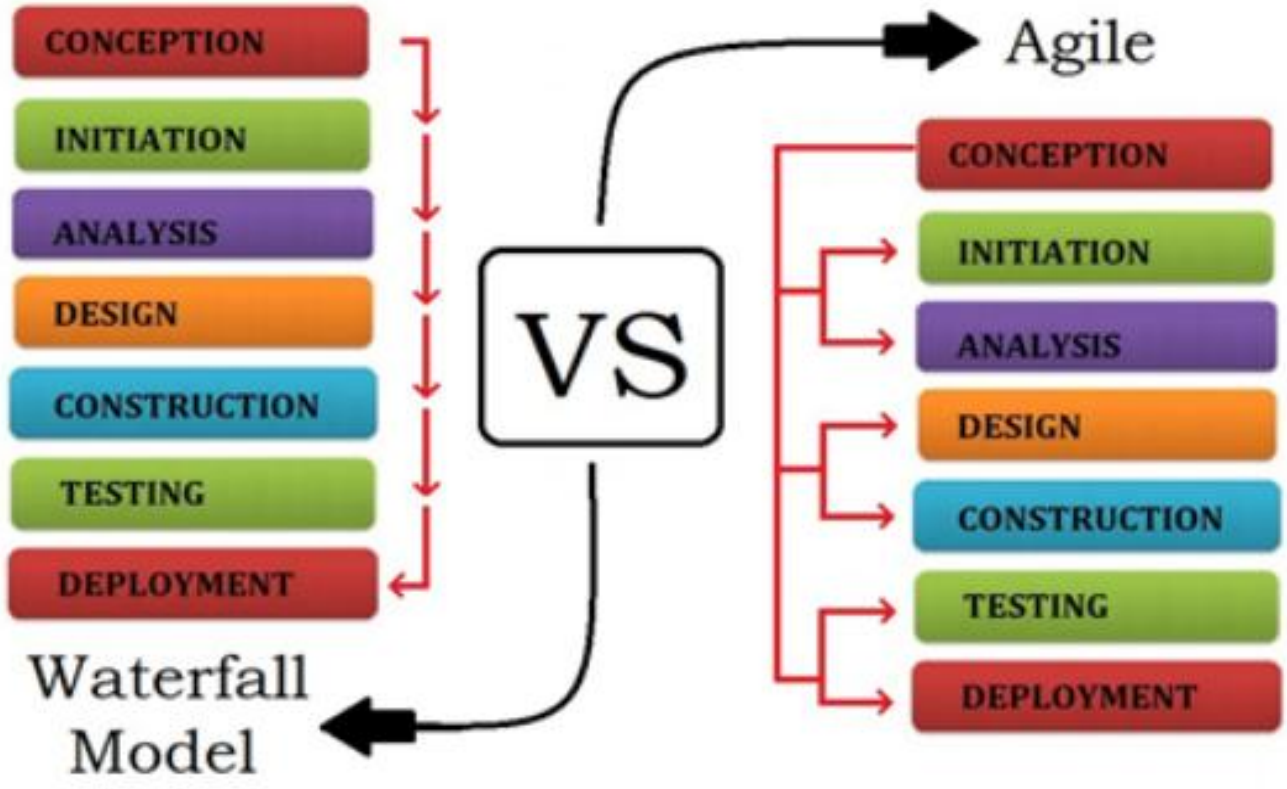


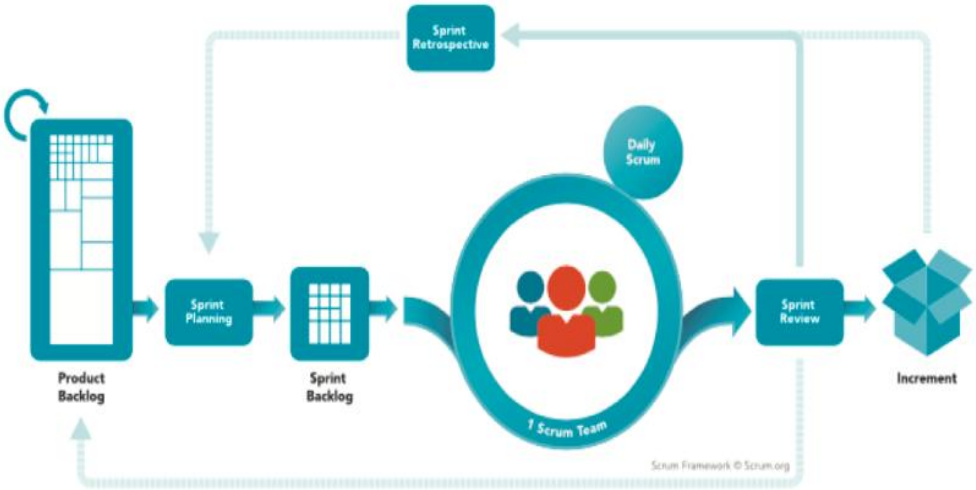
Figura 2. Metodologia tradicional x metodologia ágil

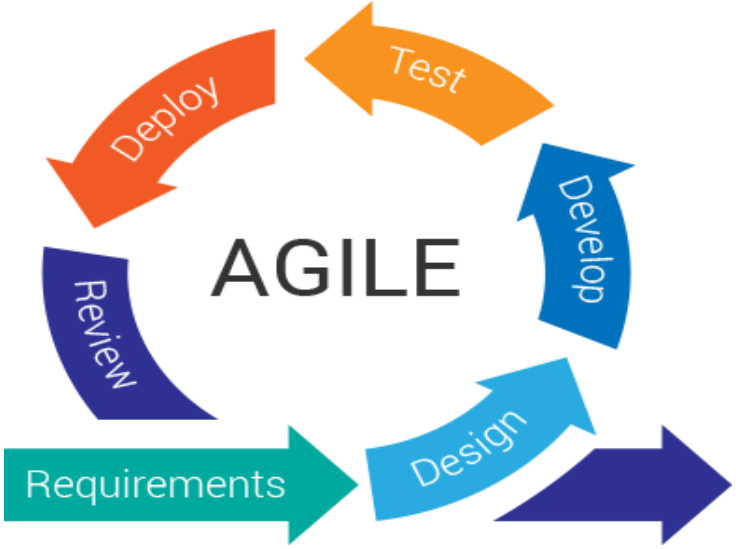
Engenharia de Software

O SCRUM é um framework ágil, iterativo e incremental.

Scrum é um framework leve que ajuda pessoas, times e organizações a gerar valor por meio de soluções adaptativas para problemas complexos. (O Guia do Scrum, 2020)

https://scrumguides.org/





Introdução do Desenvolvimento Sistemas

Definição de Full Stack

- **Full Stack é um termo utilizado para definir um profissional que possui habilidades e conhecimentos em todas camadas do desenvolvimento de sistemas, sejam eles Web, Desktop ou Mobile.**

- **Este profissional atua desde o desenvolvimento o front-end até o back-end. Eles são capazes de trabalhar com a mais diversas tecnologias e linguagens de programação.**

- **Possui um conhecimento aprofundado em todas as camadas da aplicação, o desenvolvedor Full Stack é capaz de desenvolver, projetar e implementar aplicativos e sites complexos. Eles lidam com todas as etapas do processo de desenvolvimento de software, desde a concepção até a entrega final.**

Habilidades necessárias para ser um desenvolvedor Full Stack

- **Para se tornar um desenvolvedor Full Stack, é necessário ter habilidades técnicas em diversas áreas, incluindo HTML, CSS e JavaScript para front-end, uma linguagem de programação como Java, C# ou Python para back-end, bem como habilidades em bancos de dados e infraestrutura de servidores.**

- **Além disso, é importante ter habilidades de resolução de problemas, comunicação e colaboração, já que o desenvolvimento Full Stack envolve trabalhar com equipes multidisciplinares.**

O papel do desenvolvedor Full Stack no desenvolvimento de aplicações

➢ Compreender as necessidades do cliente e traduzi-las em requisitos técnicos

- O desenvolvedor Full Stack precisa entender as necessidades do cliente e traduzi-las em requisitos técnicos.

- Envolve a compreensão de quais recursos e funcionalidades devem ser implementados no aplicativo.

O papel do desenvolvedor Full Stack no desenvolvimento de aplicações

➢ Projeto e implementação da interface do usuário

- O desenvolvedor Full Stack é responsável pelo design e implementação da interface do usuário (UI), que é a camada com a qual o usuário interage diretamente.

- Envolve habilidades em tecnologias como HTML, CSS e JavaScript.

O papel do desenvolvedor Full Stack no desenvolvimento de aplicações

Desenvolvimento do back-end e lógica de negócios

- **O desenvolvedor Full Stack também é responsável pelo desenvolvimento do back-end, que é a camada responsável pelo processamento de dados e lógica de negócios.**

- **Nesta camada é desejável o conhecimento de arquitetura de softwares e padrões de projetos.**

- **Envolve a criação de APIs e integração com bancos de dados e outras ferramentas.**

O papel do desenvolvedor Full Stack no desenvolvimento de aplicações

Gerenciamento de bancos de dados

- **O desenvolvedor Full Stack é responsável pelo gerenciamento do banco de dados, que inclui a criação de tabelas, gerenciamento de usuários e permissões, bem como a escrita de consultas para extrair dados.**

- **Envolve o conhecimento de bancos de dados relacionais e NoSQL (MongoDB, Neo4J, Redis e etc..)**

O papel do desenvolvedor Full Stack no desenvolvimento de aplicações

Teste e depuração

- O desenvolvedor Full Stack é responsável pelo teste e depuração de todas as camadas do aplicativo.
- Inclui testar a interface do usuário, verificar se a lógica de negócios está funcionando corretamente e garantir que os dados estejam sendo armazenados corretamente.

Implantação e manutenção

- O desenvolvedor Full Stack é responsável pela implantação do aplicativo em um ambiente de produção, garantindo que ele esteja configurado corretamente e funcione corretamente.

Introdução do Desenvolvimento Full Stack

Habilidades necessárias para ser um desenvolvedor Full Stack

- **Para se tornar um desenvolvedor Full Stack, é necessário ter habilidades técnicas em diversas áreas, incluindo HTML, CSS e JavaScript para front-end, uma linguagem de programação como Java, C# ou Python para back-end, bem como habilidades em bancos de dados e infraestrutura de servidores.**

- **Além disso, é importante ter habilidades de resolução de problemas, comunicação e colaboração, já que o desenvolvimento Full Stack envolve trabalhar com equipes multidisciplinares.**

CEUB
EDUCAÇÃO SUPERIOR

Habilidades necessárias para ser um desenvolvedor Full Stack

Um desenvolvedor Full Stack necessita conhecer as seguintes tecnologias:

Desenvolvimento Front-end

- **HTML, CSS, JavaScript**
- **Frameworks front-end, como React, Angular, Vue.js e Blazor**
- **Habilidade em design de interfaces de usuário (UI) e experiência do usuário (UX)**

CEUB
EDUCAÇÃO SUPERIOR

Habilidades necessárias para ser um desenvolvedor Full Stack

Um desenvolvedor Full Stack necessita conhecer as seguintes tecnologias

Desenvolvimento Back-end:

1. **Conhecimento de uma linguagem de programação:**
   - **Java e JVM (Java Virtual Machine) para desenvolvimento Java**
   - **C# e .NET Framework Core para desenvolvimento .NET**
   - **PHP para desenvolvimento PHP**
   - **Python para desenvolvimento Python**

CEUB
EDUCAÇÃO SUPERIOR

Habilidades necessárias para ser um desenvolvedor Full Stack

Um desenvolvedor Full Stack necessita conhecer as seguintes tecnologias

Desenvolvimento Back-end:

2. Frameworks back-end:

- **Spring para desenvolvimento em Java,**
- **AspNet Core para desenvolvimento em .NET (C#, F# ou VBNet)**
- **Django ou Flask para desenvolvimento Python e**
- **NEXT e NEST são dois frameworks Node.js que são comumente usados para desenvolvimento Full Stack**

CEUB
EDUCAÇÃO SUPERIOR

Habilidades necessárias para ser um desenvolvedor Full Stack

Um desenvolvedor Full Stack necessita conhecer as seguintes tecnologias:

Desenvolvimento Back-end:

3. Banco de dados:

- **SQL Server para desenvolvimento .NET**
- **MySQL, MariaDB ,PostgreSQL ou Oracle para desenvolvimento Java e PHP**
- **PostgreSQL, MySQL ou MongoDB para desenvolvimento Python**

CEUB
EDUCAÇÃO SUPERIOR

Habilidades necessárias para ser um desenvolvedor Full Stack

Um desenvolvedor Full Stack necessita conhecer as seguintes tecnologias:

Desenvolvimento Back-end:

4. Conhecimento de API RESTful:

- **Criação de APIs RESTful usando ASP.NET Web API para desenvolvimento .NET**
- **Criação de APIs RESTful usando Spring MVC para desenvolvimento Java**
- **Criação de APIs RESTful usando Flask ou Django Rest Framework para desenvolvimento Python**

CEUB
EDUCAÇÃO SUPERIOR

Habilidades necessárias para ser um desenvolvedor Full Stack

Um desenvolvedor Full Stack necessita conhecer as seguintes tecnologias:

Desenvolvimento Back-end:

5. **Habilidade em DevSecOps:**

- **Conhecimento em servidores web, como IIS ou Apache**
- **Configuração e gerenciamento de servidores, como Windows Server ou Linux**
- **Conhecimento em ferramentas de conteinerização, como Docker ou Kubernetes**
- **Habilidade em automação de infraestrutura usando ferramentas como Chef, Puppet ou Ansible**

CEUB
EDUCAÇÃO SUPERIOR

Habilidades necessárias para ser um desenvolvedor Full Stack

Um desenvolvedor Full Stack necessita conhecer as seguintes tecnologias:

Desenvolvimento Back-end:

6. Habilidade em ferramentas de desenvolvimento:

- **Git e controle de versão**
- **Teste de unidade e ferramentas de teste automatizadas**
- **Ferramentas de automação de compilação, como MSBuild para desenvolvimento .NET, Maven para desenvolvimento Java, Composer para desenvolvimento PHP e pip para desenvolvimento Python**
- **Ferramentas de gerenciamento de dependências, como NuGet para desenvolvimento .NET, Maven Central para desenvolvimento Java, Packagist para desenvolvimento PHP e PyPI para desenvolvimento Python**

CEUB
EDUCAÇÃO SUPERIOR

Habilidades necessárias para ser um desenvolvedor Full Stack

Um desenvolvedor Full Stack necessita conhecer as seguintes tecnologias:

Desenvolvimento Back-end:

7. Habilidade em ferramentas de desenvolvimento:

- **Git e controle de versão**
- **Teste de unidade e ferramentas de teste automatizadas**
- **Ferramentas de automação de compilação, como MSBuild para desenvolvimento .NET, Maven para desenvolvimento Java, Composer para desenvolvimento PHP e pip para desenvolvimento Python**
- **Ferramentas de gerenciamento de dependências, como NuGet para desenvolvimento .NET, Maven Central para desenvolvimento Java, Packagist para desenvolvimento PHP e PyPI para desenvolvimento Python**

CEUB
EDUCAÇÃO SUPERIOR

Ferramentas para Desenvolvimento Back-end:

Um bom desenvolvedor Full Stack deve conhecer IDEs (Integrated Development Environment) para o desenvolvimento de suas atividades.

IDEs são ferramentas de desenvolvimento que permitem escrever, depurar e testar código em uma única interface.

IDEs mais utilizadas:

- **Visual Studio (para desenvolvimento .NET)**
- **Eclipse (para desenvolvimento Java),**
- **PyCharm (para desenvolvimento Python) e**
- **IntelliJ IDEA (para desenvolvimento Java, Kotlin e outras linguagens).**
- **Android Studio para desenvolvimento Mobile**

Ferramentas para Desenvolvimento Back-end:

Editores de código:

Editores de código são ferramentas de desenvolvimento mais leves do que os IDEs, que fornecem recursos básicos de edição de código, como destaque de sintaxe, auto-completar e indentação.

Alguns dos editores de código mais populares incluem:

- **Visual Studio Code,**
- **Sublime Text e**
- **Atom.**

Exemplos de aplicações Full Stack

As aplicações Full Stack são amplamente utilizadas em diversos setores, incluindo o comércio, o governo e a indústria.

- **Sistemas de gestão pública**: São aplicações full stack utilizadas por governos para gerenciar processos e serviços públicos, como sistemas de controle de gastos, licitações, contratos, recursos humanos, entre outros.
- **Redes sociais**: plataformas como Facebook, Twitter e Instagram são exemplos de aplicações full stack.
- **E-commerce**: lojas virtuais como Amazon, Mercado Livre e Submarino são exemplos de aplicações full
- **Sistemas de gerenciamento de relacionamento com o cliente** (CRM) e de recursos empresariais (ERP), como Salesforce e HubSpot, também são comuns em aplicações comerciais.
- **Etc...**

Importância do desenvolvimento Full Stack

O desenvolvimento Full Stack é de extrema importância no cenário atual, pois permite que os desenvolvedores possam criar aplicações completas, desde o back-end até o front-end, garantindo a qualidade e a eficiência de todo o sistema.

Com o aumento da demanda por aplicações digitais e com a sua crescente complexidade, o desenvolvimento Full Stack se torna ainda mais relevante, pois oferece uma visão mais ampla sobre a arquitetura e o funcionamento do sistema como um todo.

Importância do desenvolvimento Full Stack

1. **Necessidade de desenvolvimento de Aplicações Completas**
   - O desenvolvimento Full Stack é de extrema importância no cenário atual, pois permite que os desenvolvedores criar aplicações completas, desde o back-end até o front-end, garantindo a qualidade e a eficiência de todo o sistema.
2. **Aumento da demanda por aplicações digitais com nível de complexidade crescente:**
   - Neste cenário o desenvolvimento Full Stack se torna ainda mais relevante, pois oferece uma visão mais ampla sobre a arquitetura e o funcionamento do sistema como um todo.

Importância do desenvolvimento Full Stack

3. **Integração de tecnologias com Inteligência Artificial (IA)**
   - **A integração de tecnologias com Inteligência Artificial (IA) nas aplicações Full Stack permite a criação de soluções mais avançadas e personalizadas, capazes de aprender com os dados e se adaptar às necessidades dos usuários.**

   - **A IA pode ser utilizada em diversas áreas, como segurança, saúde, finanças, marketing, entre outras, contribuindo para a criação de soluções mais eficientes e inovadoras.**

Importância do desenvolvimento Full Stack

4. Agilidade no desenvolvimento de aplicações

- O desenvolvimento Full Stack também permite uma maior agilidade no desenvolvimento de aplicações, já que o desenvolvedor é capaz de criar e testar todo o sistema de forma integrada, sem a necessidade de depender de terceiros para implementar funcionalidades específicas.

- Isso acelera o processo de desenvolvimento e permite que as aplicações sejam lançadas mais rapidamente no mercado.

Tendências e Tecnologias em Full Stack

Existem várias tendências e tecnologias em Full Stack que estão em alta atualmente.

Algumas das principais são:

- **Microserviços - uma abordagem para arquitetura de software que divide o sistema em serviços independentes, tornando-o mais escalável e fácil de manter.**

- **API-first - uma abordagem de desenvolvimento de software que prioriza a criação de interfaces de programação de aplicativos (APIs) antes de construir a interface do usuário, o que facilita a integração com outros sistemas e plataformas.**

CEUB
EDUCAÇÃO SUPERIOR

Tendências e Tecnologias em Full Stack

- **Inteligência artificial e aprendizado de máquina** - tecnologias que permitem aos desenvolvedores criar aplicações que possam aprender e se adaptar com o tempo.

- **Cloud Computing –** Número crescente de empresas que estão adotando soluções de computação em nuvem, como Amazon Web Services (AWS), Microsoft Azure e Google Cloud Platform, para hospedar suas aplicações e serviços.

- **Serverless** - É uma arquitetura em que a aplicação é executada em um ambiente sem servidor, gerenciado pelo provedor de nuvem. Isso permite uma maior escalabilidade, flexibilidade e redução de custos, já que o provedor de nuvem é responsável por gerenciar a infraestrutura.

Tendências e Tecnologias em Full Stack

- **DevSecOps** - uma cultura que promove a colaboração entre desenvolvedores e equipes de operações para fornecer software de maneira mais rápida e confiável.

- **Low-Code** - Plataformas que permitem aos desenvolvedores criar aplicativos sem escrever código, usando uma interface visual. Isso pode reduzir significativamente o tempo e o esforço necessários para criar aplicativos.

Conceitos básicos de arquitetura cliente-servidor

Arquitetura Cliente Servidor

A arquitetura cliente-servidor é a arquitetura mais citada quando nós falamos em sistemas distribuídos. Ela é historicamente a mais importante e a mais amplamente empregada (COULOURIS, 2013).

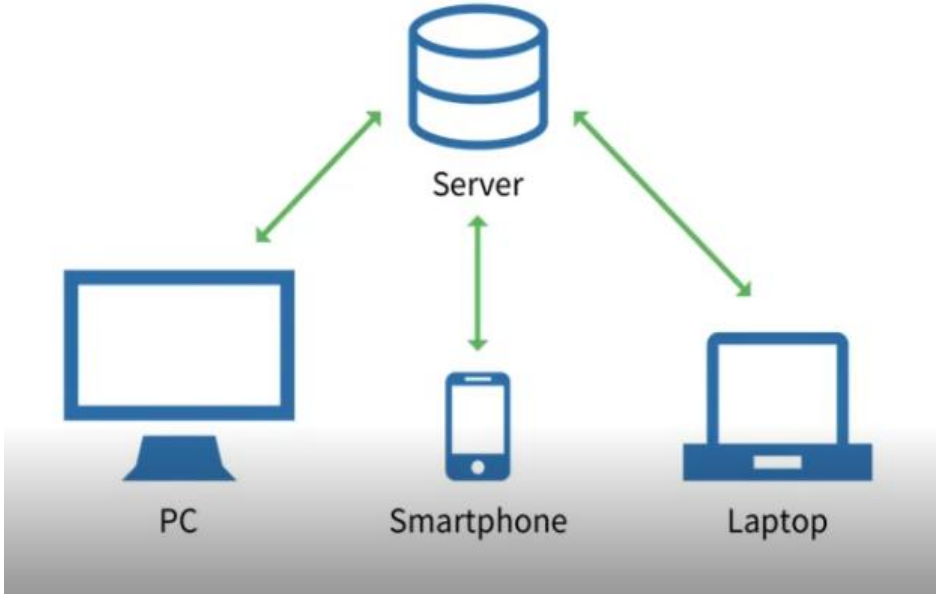
Processos são classificados de acordo com as responsabilidades de cada um.

- **Processo servidor é um processo que implementa um serviço específico – por exemplo, um serviço de sistema de arquivos.**
- **Um processo cliente é um processo que requisita um serviço de um servidor enviando-lhe uma requisição e, na sequência, esperando pela resposta do servidor.**

Arquitetura Cliente Servidor

Arquitetura cliente-servidor ou modelo cliente-servidor é uma arquitetura na qual o processamento da informação é dividido em módulos ou processos distintos.

Existe um processo que é responsável pela manutenção da informação (servidores) e outro responsável pela obtenção dos dados (os clientes).



Arquitetura Cliente Servidor

Na arquitetura cliente-servidor, a comunicação entre o cliente e o servidor é feita por meio de protocolos de rede, como HTTP ou TCP/IP.

O cliente envia uma solicitação ao servidor, especificando qual operação deve ser realizada (por exemplo, buscar informações em um banco de dados).

O servidor recebe a solicitação, executa a operação correspondente e envia a resposta de volta para o cliente, geralmente na forma de uma página web, um conjunto de dados ou um arquivo.

CEUB
EDUCAÇÃO SUPERIOR

Arquitetura Cliente Servidor

Servidor

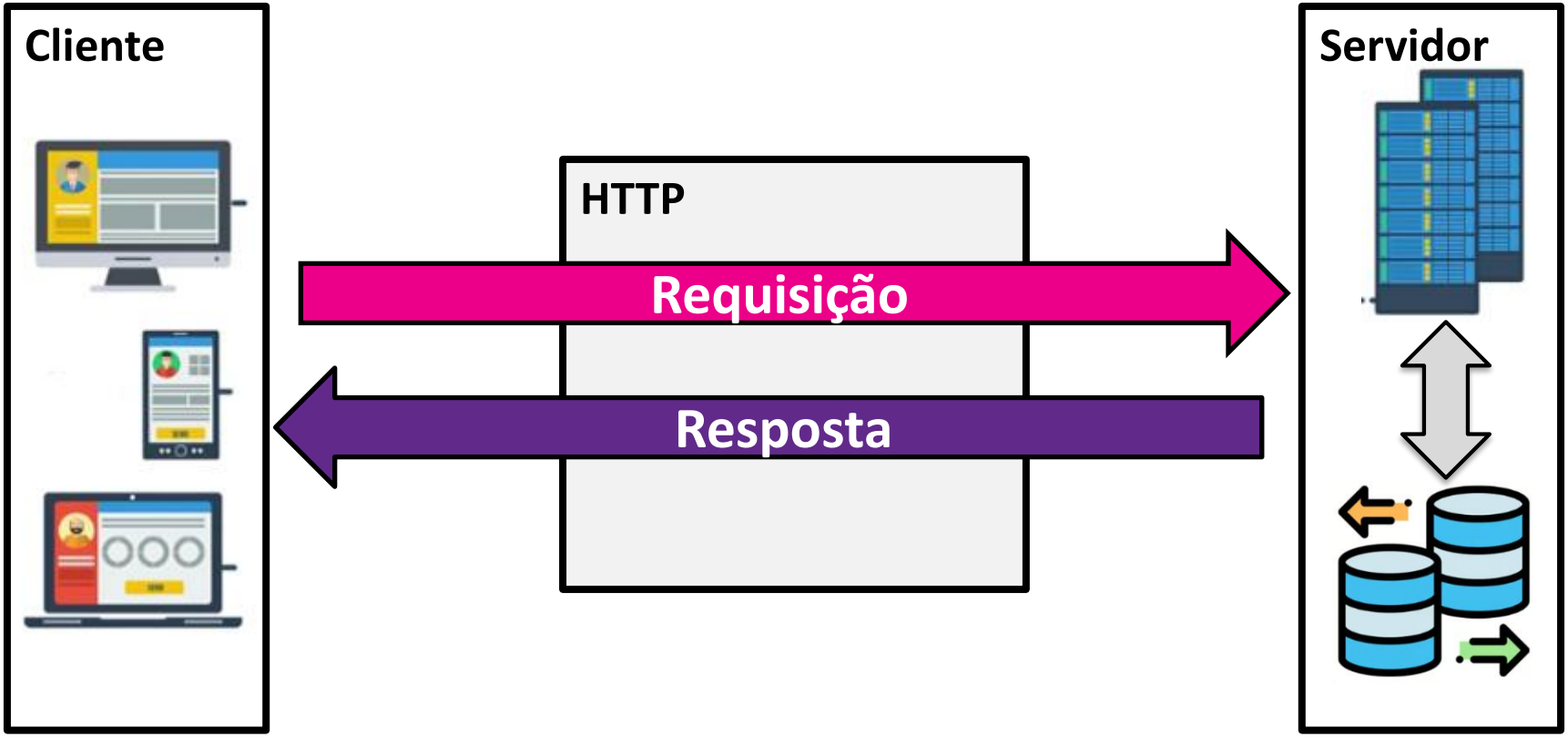
- Recebe e responde a solicitações dos clientes;
- Atende a diversos clientes simultaneamente;
- Exige hardware mais robusto, com maior capacidade de processamento;
- Software sempre em execução, aguardando ser chamado pelo cliente.

Cliente

- Inicia e termina a comunicação com o servidor;
- Não necessita de hardware especial;
- Interação com o usuário final.

Arquitetura Cliente Servidor

Comunicação entre o cliente e o servidor é feita por meio de protocolos de rede, como HTTP ou TCP/IP



Desenvolvimento Web

Introdução ao desenvolvimento web

O que é desenvolvimento web

- **Definição de desenvolvimento web: o processo de criação de aplicações e conteúdos que podem ser acessados pela internet, utilizando tecnologias como HTML, CSS, JavaScript e outras.**
- **Diferença entre desenvolvimento web e desenvolvimento de software tradicional**
- **Importância do desenvolvimento web na era digital**

Introdução ao desenvolvimento web

História do desenvolvimento web

- Evolução do desenvolvimento web desde a criação da World Wide Web por Tim Berners-Lee em 1989
- Diferentes fases do desenvolvimento web, incluindo a era dos sites estáticos, a era dos sites dinâmicos, a era dos aplicativos web e a era dos aplicativos móveis
- Principais marcos na história do desenvolvimento web, incluindo a criação do primeiro navegador web, a criação do primeiro servidor web e o lançamento do primeiro smartphone

CEUB
EDUCAÇÃO SUPERIOR

Introdução ao desenvolvimento web

Conceito de Internet e Web.

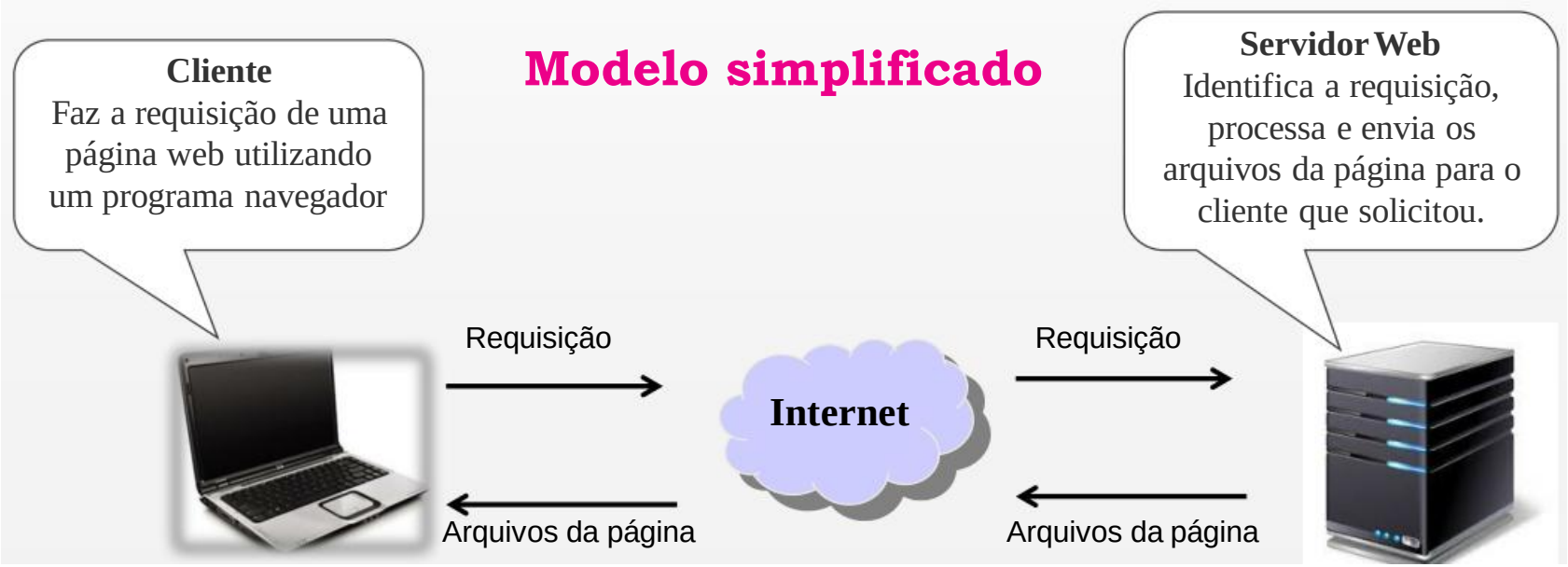
E, embora muitos utilizem os dois termos (web e internet) de forma indiscriminada, eles não são sinônimos.

Quando falamos de internet, estamos nos referindo a uma grande rede de dispositivos computadorizados de alcance mundial, podemos entendê-la como uma grande infraestrutura em rede.

A web (uma derivação abreviada para a expressão World Wide Web) é apenas uma das funcionalidades da internet – no caso específico, navegar através do hipertexto.

Servidor Web

O termo Servidor Web é frequentemente utilizado para fazer referência ao sistema de software que recebe solicitações de páginas de Internet através do protocolo HTTP e processa e envia os dados solicitados.
O termo também pode ser utilizado para designar o computador servidor que executa o software descrito acima.



Introdução

Funcionamento da Internet –Protocolo HTTP

HTTP é um protocolo de transferência que possibilita que as pessoas que inserem a URL do seu site na Web possam ver os conteúdos e dados que nele existem. A sigla vem do inglês Hypertext Transfer Protocol.

O protocolo HTTP é de particular interesse, pois será utilizado como base para a implementação de um sistema para a World Wide Web, também denominada simplesmente Web ou WWW, que trata de todo o grande conjunto de documentos de hipermídia disponíveis e interligados existentes na Internet.

Protocolo HTTP

O protocolo HTTP é o principal meio de troca de mensagens de hipermídia para a Web no paradigma cliente servidor (requisição e resposta), pois seu objetivo é prover a comunicação entre uma aplicação cliente (como o browser) e uma aplicação servidor (hospedagem de websites e serviços), possibilitando ao cliente realizar requisições (request) ao servidor e o este retornar uma resposta (response) ao cliente.

WEB SITE OU APLICAÇÃO WEB

Web site

- Um Web site é uma coleção de páginas HTML estáticas, ou seja, que não interagem com um banco de dados através de uma linguagem de servidor Web.

Aplicação Web

- Uma aplicação Web também pode conter uma coleção de páginas, porém o conteúdo destas páginas é montado dinamicamente, ou seja, é carregado através de solicitações (requisições).
- Uma aplicação Web é mais complexa porque precisa de uma linguagem de servidor para poder intermediar as solicitações do navegador. Exemplo de linguagens de servidor Web: C#, Java, PHP, Python etc.

Tecnologias Utilizadas Para o Desenvolvimento Web

Linguagens de Programação – Back-End

Não há como dizer qual é a melhor, pois para o desenvolvimento Web temos muitas opções:

- **.NET;**
- **Java;**
- **Python;**
- **PHP;**
- **Perl.**
- **Ruby;**

Na verdade você deve aprender todas e ser muito bom em pelo menos uma delas.

Tecnologias client-side

Tecnologias client-side são tecnologias utilizadas no desenvolvimento web do lado cliente (parte da aplicação que roda no navegador),

Frameworks e bibliotecas para desenvolvimento web

ASP.NET: É um framework da Microsoft para desenvolvimento web usando a linguagem C#. É uma plataforma completa para construir aplicativos web e APIs, com recursos como autenticação, autorização, cache, roteamento, gerenciamento de sessões e muito mais.

Spring: É um framework em Java para construir aplicativos web empresariais. Possui recursos como injeção de dependência, ORM, autenticação, autorização, gerenciamento de sessões e muito mais.

Django: É um framework web em Python que permite desenvolver rapidamente aplicações web robustas e seguras. Inclui recursos como ORM, autenticação, roteamento, gerenciamento de sessões, administração de site e muito mais.

CEUB
EDUCAÇÃO SUPERIOR

Frameworks e bibliotecas para desenvolvimento web

React é um biblioteca JavaScript criado pelo Facebook para desenvolvimento de interfaces de usuário, utilizando uma abordagem baseada em componentes.

Angular é um framework JavaScript mantido pela Google, que oferece recursos para desenvolvimento de aplicativos web escaláveis, usando o padrão MVC (Model-View-Controller).

Vue é um framework JavaScript de código aberto, que permite construir interfaces de usuário reativas e componentes reutilizáveis.

Blazor é um framework web da Microsoft, que permite desenvolver aplicativos web interativos e de alta performance utilizando a linguagem C# e o .NET. Ele utiliza WebAssembly para executar o código C# no navegador.

CEUB
EDUCAÇÃO SUPERIOR

Frameworks e bibliotecas para desenvolvimento web

Next.js

Next.js é um framework de desenvolvimento web de código aberto baseado em React, que permite a criação de aplicativos da web do lado do servidor (SSR) e do lado do cliente (CSR).

Foi criado para tornar a construção de aplicativos da web com React mais fácil e eficiente, fornecendo recursos avançados, como roteamento de páginas automático, pré-renderização e geração estática de páginas.

Desenvolvimento Mobile

Desenvolvimento para Dispositivos móveis

O desenvolvimento de dispositivos móveis envolve a criação de aplicativos para dispositivos móveis, como smartphones e tablets.

Os principais conceitos relacionados ao desenvolvimento de dispositivos móveis incluem:

- **Escolha da plataforma (Android ou iOS),**
- **Design responsivo,**
- **Desenvolvimento nativo ou híbrido,**
- **Armazenamento de dados,**
- **Segurança, testes de qualidade, integração de APIs, Localização e internacionalização e a acessibilidade.**

Todos esses conceitos são importantes para garantir que os aplicativos móveis sejam funcionais, eficientes e seguros para os usuários.

Tecnologias de Desenvolvimento para Dispositivos móveis

Principais plataformas de desenvolvimento: Tecnologias Alternativas

Comparativo entre as frameworks para desenvolvimento híbrido

Attribute	React Native	Xamarin		IONIC	Flutter
Programming Language	JavaScript + Swift,Objective-C or Java	C# with .net environment		HTML5, CSS, and JavaScript + Typescript	Dart
Performance	Close-to-native ★★★½	Xamarin ios/Android Close-to-native ★★★★	Xamarin Forms Moderate ★★★	Moderate ★★	Amazing ★★★★★
GUI	Use Native UI Controllers	Use Native UI Controllers		HTML,CSS	Use Proprietary Widgets and deliver amazing UI
Market and Community Support	Very Strong	Strong		Strong	Not very popular
Use Cases	All apps	Simple apps		Simple apps	All apps
Code Reusability	90% of code is reusable	96% of code is reusable		98% of code is reusable	50-90% (approx.)of code is reusable
Popular Apps	Facebook, Instagram, Airbnb, UberEats	Olo, the World Bank, Storyo.		JustWatch, Pacifica, and Nationwide.	HamiltTon
Pricing	Open-source	Open-source + Paid as well		Open-source + Paid as well	Open-source

Diferença entre desenvolvimento nativo e multiplataforma

Desenvolvimento Nativo

Os aplicativos nativos usam a tecnologia/linguagem da plataforma, ou seja, Objective C ou Swift para iOS e Java ou Kotlin para Android.

Os aplicativos nativos aproveitam totalmente a plataforma subjacente, eles fornecem o melhor desempenho e experiência do usuário.

No entanto, ao desenvolver aplicativos para várias plataformas em sua base de código nativa, você deve desenvolver o aplicativo iOS e Android separadamente. Isso significa que você precisa de duas bases de código diferentes e, geralmente, de duas equipes de desenvolvimento diferentes.

Diferença entre desenvolvimento nativo e multiplataforma

Desenvolvimento Multiplataforma

Os aplicativos de cross-plataform ou seja de plataforma cruzada são escritos em uma linguagem diferente (diferente da linguagem do sistema operacional) e renderizam uma experiência de aplicativo nativa em tempo de execução.

A vantagem dos aplicativos de plataforma cruzada é que eles podem ser desenvolvidos com uma única base de código, economizando muito tempo, esforço e custo. Você pode aproveitar algumas das linguagens de codificação mais populares sem ter que aprender Swift, Objective C, Java ou Kotlin.

Esses aplicativos apresentam um desempenho inferior quando comparados aos aplicativos nativos, mas a diferença de desempenho é superada pela facilidade de desenvolvimento.

Segurança no desenvolvimento de software

O que é Segurança no Desenvolvimento de Software?

É o conjunto de práticas, processos e ferramentas utilizados para proteger o software contra vulnerabilidades, ataques e uso indevido durante todo o seu ciclo de vida.

Mudança de Paradigma: Segurança não é mais apenas responsabilidade da infraestrutura (firewall/redes), mas começa na **linha de código**.

Segurança no desenvolvimento de software

Objetivo: Garantir que o software continue funcionando corretamente mesmo sob ataque malicioso.

Custo do Erro: Corrigir uma falha na fase de design(projeto) custa até 30x menos do que corrigir após o lançamento (produção).

Pilares da Segurança da Informação

Confidencialidade (Sigilo):

Garantia de que a informação só é acessível a quem tem autorização.

Exemplo: Criptografia de senhas e dados sensíveis.

Pilares da Segurança da Informação

Integridade (Consistência):

Garantia de que a informação não foi alterada indevidamente (por erro ou ataque).

Exemplo: Logs de auditoria e Hashing de arquivos.

Pilares da Segurança da Informação

Disponibilidade (Acesso):

Garantia de que o sistema está operacional para usuários legítimos quando necessário.

Exemplo: Proteção contra ataques de negação de serviço (DDoS).

Security by Design e o Ciclo de Vida (S-SDLC)

Security by Design (Segurança desde o projeto):

A segurança é um requisito funcional, pensado na arquitetura antes de digitar o código.

Shift-Left Security:

Mover os testes de segurança para o início do processo (esquerda da linha do tempo), realizando testes durante o desenvolvimento, não apenas no final.

Security by Design e o Ciclo de Vida (S-SDLC)

Modelagem de Ameaças:

Identificar potenciais vetores de ataque antes de construir a funcionalidade.

DevSecOps:

Integração contínua de desenvolvimento, segurança e operações.

Controle de versão com Git:Estratégias de Branching: Organizando o Caos

Por que criar branches?

Permite trabalhar em novas funcionalidades (features) ou correções sem quebrar o código principal que já funciona.

A Estrutura Padrão:

- **main (ou master): Código de produção. Sagrado, estável e testado.**
- **develop: Ambiente de integração. Onde o trabalho da equipe se junta antes de ir para produção.**
- **feature/*: Branches temporárias para criar algo novo (ex: feature/login-page).**
- **hotfix/*: Correções urgentes para bugs em produção.**

Regra de Ouro: Nunca commitar diretamente na main sem revisão.

Obrigado